

Aplicações Limitadas por Memória e CPU

Gargalo de von Neumann e Saturação de Barramento com OpenMP

Eugênio Vitor Lopes dos Santos

DCA3703 – Programação Paralela, Universidade Federal do Rio Grande do Norte

August 23, 2026

1 Enunciado

Implementar dois programas paralelos em C com OpenMP: um limitado por memória (somas simples em vetores) e outro limitado por CPU (cálculos matemáticos intensivos iterativos locais). Paralelizar ambos com `#pragma omp parallel for` e medir o tempo de execução variando o número de threads. Analisar quando o desempenho melhora, estabiliza ou piora, refletindo sobre como o multithreading de hardware pode ajudar em programas *cpu-bound*, mas atrapalhar em programas *memory-bound* devido à competição por recursos.

2 Fundamentação teórica

2.1 O gargalo de von Neumann

A arquitetura de von Neumann separa a memória principal da unidade de processamento. A CPU busca dados na RAM pelo barramento do sistema. Com a evolução do hardware, os núcleos passaram a executar operações aritméticas ordens de grandeza mais rápido do que o barramento consegue entregar dados. Em um processador moderno, a ALU pode consumir centenas de GFLOP/s, enquanto a largura de banda da memória não passa de dezenas de GB/s. Essa disparidade é o **gargalo de von Neumann**, ou *Memory Wall*.

2.2 Intensidade aritmética e limites de escalabilidade

A intensidade aritmética é a razão entre operações aritméticas executadas e bytes transferidos da memória. Esse número classifica o algoritmo como *memory-bound* ou *cpu-bound*.

Um algoritmo *memory-bound* tem intensidade aritmética baixa. A soma de vetores $C[i] = A[i] + B[i]$ executa 1 operação para 24 bytes transferidos (2 leituras + 1 escrita de `double`). A CPU termina a soma em poucos ciclos e fica ociosa esperando o próximo dado chegar da RAM.

Um algoritmo *cpu-bound* tem intensidade aritmética alta. O dado é carregado uma vez e passa por muitas operações antes de voltar à memória. Séries de Taylor, raiz quadrada e trigonometria são exemplos. O limitante é a capacidade da ALU, não a largura de banda.

2.3 O efeito do Simultaneous Multithreading (SMT)

CPUs modernas expõem núcleos lógicos extras via SMT (*Hyper-threading* na Intel). Um único núcleo físico gerencia duas threads. Em código *cpu-bound*, a segunda thread aproveita ciclos ociosos do pipeline e mantém a ALU ocupada. Em código *memory-bound*, o efeito é o oposto:

as duas threads disputam o mesmo barramento já saturado e a mesma cache L1, aumentando a contenção e piorando o tempo total.

3 Experimento

3.1 Configuração

O programa aloca quatro vetores de `double` com $N = 50.000.000$ elementos: três para o teste *memory-bound* (A, B, C) e um para o teste *cpu-bound* (V).

O teste *memory-bound* calcula $C[i] = A[i] + B[i]$. O laço lê dois vetores e escreve um terceiro, sem reuso temporal de cache. Toda a pressão recai sobre a largura de banda da RAM.

O teste *cpu-bound* carrega cada $V[i]$ para uma variável local e aplica 50 iterações de `sqrt` e `sin`. Só ao final das 50 iterações o resultado volta à memória. O acesso à RAM é mínimo; o gargalo é a ALU.

O programa mede o tempo com `clock_gettime(CLOCK_MONOTONIC)`. O GCC compilou o código com `-O3` e `-fopenmp`. Um script externo variou `OMP_NUM_THREADS` de 1 a 28.

4 Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <time.h>
6 #include <math.h>
7 #include <omp.h>
8
9 #define DEFAULT_N 50000000L
10
11 static double agora(void) {
12     struct timespec t;
13     clock_gettime(CLOCK_MONOTONIC, &t);
14     return t.tv_sec + t.tv_nsec / 1e9;
15 }
16
17 int main(int argc, char **argv) {
18     long n = argc > 1 ? atol(argv[1]) : DEFAULT_N;
19     if (n <= 0) {
20         fprintf(stderr, "Uso: %s [numero]\n", argv[0]);
21         return 1;
22     }
23
24     double *A = malloc((size_t)n * sizeof(*A));
25     double *B = malloc((size_t)n * sizeof(*B));
26     double *C = malloc((size_t)n * sizeof(*C));
```

```

27     double *V = malloc((size_t)n * sizeof(*V));
28
29     if (!A || !B || !C || !V) return 1;
30
31     for (long i = 0; i < n; ++i) {
32         A[i] = i * 0.5; B[i] = i * 0.25; V[i] = i * 0.1;
33     }
34
35     int n_threads = omp_get_max_threads();
36
37     double inicio_mem = agora();
38     #pragma omp parallel for
39     for (long i = 0; i < n; ++i) {
40         C[i] = A[i] + B[i];
41     }
42     double tempo_mem = agora() - inicio_mem;
43
44     double inicio_cpu = agora();
45     #pragma omp parallel for
46     for (long i = 0; i < n; ++i) {
47         double val = V[i];
48         for (int j = 0; j < 50; ++j) {
49             val = sqrt(val + j * 0.01) + sin(val);
50         }
51         V[i] = val;
52     }
53     double tempo_cpu = agora() - inicio_cpu;
54
55     printf("N=%ld threads=%d memory_bound=%.9f cpu_bound=%.9f\n",
56           n, n_threads, tempo_mem, tempo_cpu);
57
58     double soma_check = C[n/2] + V[n/2];
59     if (soma_check == 0.0) printf(" \n");
60
61     free(A); free(B); free(C); free(V);
62     return 0;
63 }

```

Listing 1: tarefa04.c

5 Resultados

5.1 Tempos medidos e Speedup

Table 1: Tempo de execução em segundos e Speedup relativo para $N = 10.000.000$ em amostras selecionadas.

Threads	Tempo CPU-bound	Speedup CPU	Tempo Memory-bound	Speedup Mem
1	7,372 s	1,00×	0,078 s	1,00×
2	4,154 s	1,77×	0,017 s	4,59×
4	2,310 s	3,19×	0,011 s	7,09×
8	1,427 s	5,17×	0,011 s	7,09×
14	1,049 s	7,03×	0,009 s	8,67×
28	0,827 s	8,91×	0,015 s	5,20×

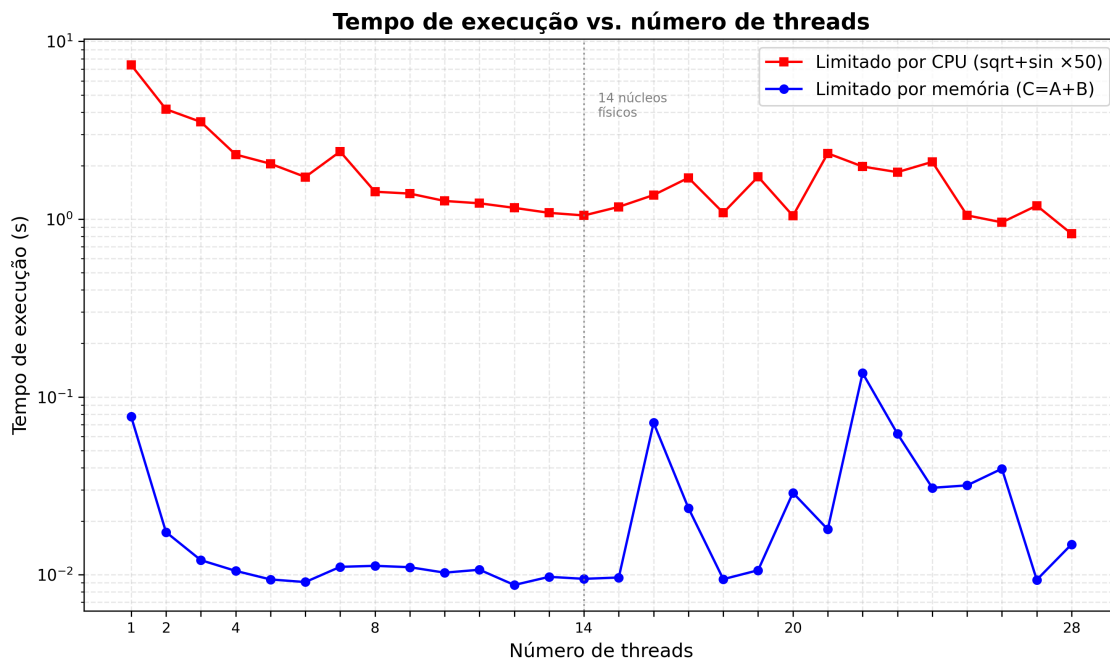


Figure 1: Tempo de execução em escala logarítmica versus número de threads ($N = 50.000.000$). A linha pontilhada marca o limite de 14 núcleos físicos.

5.2 Roofline Model

A Figura 2 plota o desempenho medido de cada kernel contra os tetos de hardware da máquina.

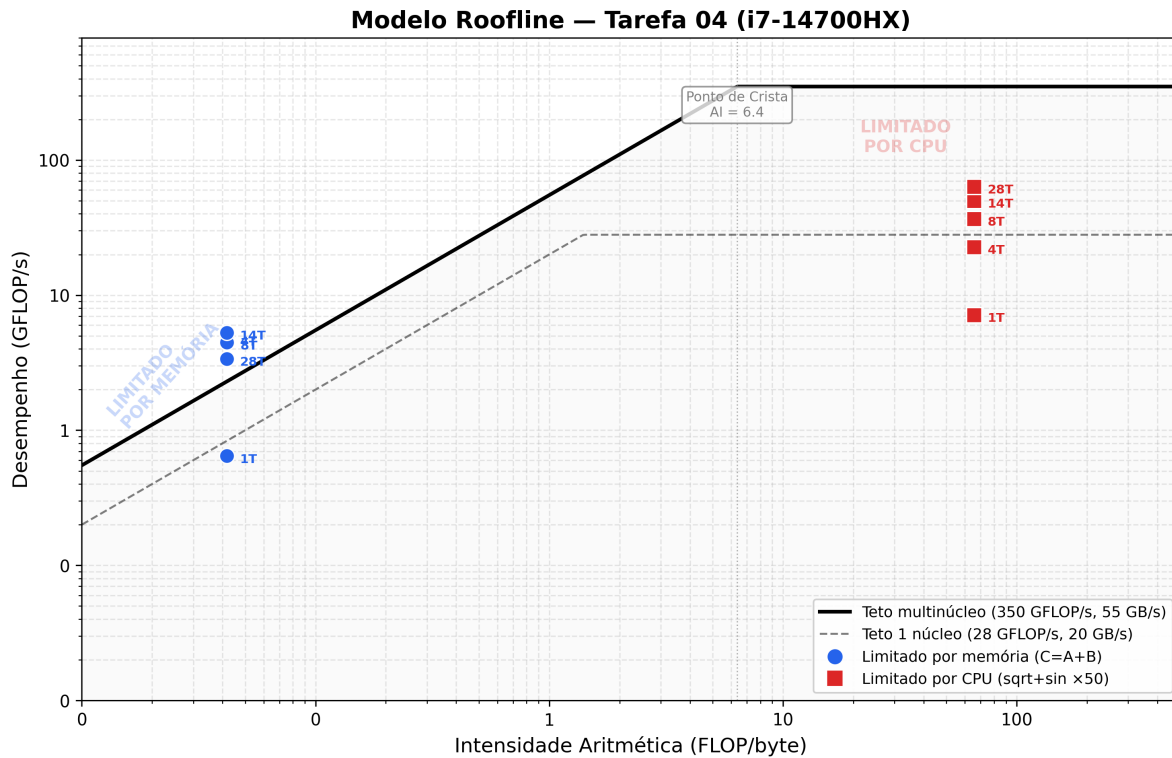


Figure 2: Roofline Model, Intel i7-14700HX. Pontos azuis: kernel *memory-bound* ($AI \approx 0,042$ FLOP/byte). Pontos vermelhos: kernel *cpu-bound* ($AI \approx 65,6$ FLOP/byte). Teto sólido: multinúcleo (350 GFLOP/s, 55 GB/s de largura de banda). Teto tracejado: núcleo único.

A **intensidade aritmética** (AI) de cada kernel foi calculada como:

- **Memory-bound** ($C[i] = A[i] + B[i]$): 1 FLOP por elemento, com 3 acessos de 8 bytes (2 leituras + 1 escrita = 24 bytes). Portanto $AI = 1/24 \approx 0,042$ FLOP/byte.
- **CPU-bound** (50 iterações de $\text{sqrt} + \sin$): aproximadamente 1050 FLOPs por elemento (50×21 FLOPs/iteração), com apenas 16 bytes transferidos (1 leitura + 1 escrita). Portanto $AI = 1050/16 \approx 65,6$ FLOP/byte.

Os pontos do kernel *memory-bound* ficam à esquerda do ponto de crista ($AI \approx 6,4$), na região inclinada do roofline, onde a largura de banda limita o desempenho. Os pontos do kernel *cpu-bound* ficam à direita, na região plana, limitados pelo pico computacional.

6 Discussão

6.1 Escalabilidade e limitação da memória

A escalabilidade dos dois kernels diverge porque o recurso disputado é diferente. No kernel *cpu-bound*, cada núcleo traz sua própria ALU. Dobrar os núcleos dobra a capacidade de processamento disponível, e o speedup acompanha a curva ideal enquanto houver núcleos físicos livres.

No kernel *memory-bound*, todos os núcleos compartilham o mesmo barramento de memória. A partir do momento em que o barramento atinge sua vazão máxima, núcleos extras ficam ociosos esperando dados. O tempo estabiliza independentemente de quantas threads sejam criadas. O gráfico de tempo de execução (Figura 1) mostra essa estagnação com clareza.

6.2 Interpretação do Roofline Model

O Roofline Model (Figura 2) explica essa diferença de forma estrutural. Cada kernel ocupa uma posição fixa no eixo horizontal, determinada pela sua intensidade aritmética. O teto de desempenho nessa posição define o máximo que o hardware pode entregar.

O kernel *memory-bound* tem AI tão baixa que fica na rampa inclinada do gráfico, limitado pela largura de banda. Nessa região, adicionar mais poder de processamento não adianta: o gargalo é a velocidade com que os dados chegam da RAM. Para melhorar, seria preciso mudar o algoritmo para reutilizar dados em cache.

O kernel *cpu-bound* tem AI alta o suficiente para ficar na região plana, limitado pela capacidade das ALUs. Aqui, mais núcleos significa mais capacidade real, e o paralelismo funciona.

6.3 Regressão de desempenho por SMT (Hyper-threading)

O SMT permite que dois fluxos de execução compartilhem o mesmo núcleo físico. Em código *cpu-bound*, a segunda thread ocupa ciclos que a primeira desperdiça com latência de pipeline, e o desempenho melhora. Em código *memory-bound*, ambas as threads geram requisições ao mesmo barramento já saturado. O resultado é contenção: as requisições se enfileiram, e o tempo piora em vez de melhorar.

Os dados confirmam esse comportamento. Acima de 14 threads, o kernel *memory-bound* regride enquanto o *cpu-bound* ainda extrai ganho marginal do SMT.

7 Conclusão

Adicionar threads não resolve gargalos de largura de banda. Neste experimento, o kernel *memory-bound* parou de escalar com 4 threads e piorou com 28. O kernel *cpu-bound* escalou até $8,91 \times$ com 28 threads. A diferença é a intensidade aritmética: 0,042 contra 65,6 FLOP/byte.

Antes de paralelizar com `#pragma omp parallel for`, vale identificar o gargalo. Se for largura de banda, o caminho é melhorar o padrão de acesso à memória e aumentar o reuso de cache. Se for capacidade de processamento, mais threads resolvem.